

Code description for the Autonomous Vehicle Storage and Retrieval System (AVS/RS) Design Space Exploration using NSGA-II and PyCh

1 Contents

- `solution.py`: Main script that orchestrates the exploration process and the optimization procedure.
- `config.py`: Configuration file for system parameters.
- `genetic_algorithm.py`: Implementation of the genetic algorithm functions (mutation, recombination, evaluation, and selection) along with its utilities.
- `AVSRS.py`: PyCh model of the AVS/RS used for simulation and evaluation.
- `control_policies.py`: Defines the control policies utilized in the simulation.
- `PyCh_architecture_diagram.pdf`: Architecture diagram of the PyCh model.
- `plot.py`: Functions to plot figures from the `.csv` files.

2 Installation

The Python scripts can be executed using any Python IDE or directly from the command line. The code was developed and tested using Python version 3.13.7. Make sure to have a compatible version installed on your system. Python can be downloaded from: www.python.org/downloads/.

PyCh requires the following packages: `simpy`, `numpy`, `matplotlib.pyplot` and `dataclasses`. To set up the environment and install PyCh, create a new virtual environment and run the following commands:

```
python -m venv .venv
.venv\Scripts\activate
pip install -e pych-source-code-main/
```

3 Usage

1. Place all downloaded files in the same folder.
2. Configure the desired parameters in the `config.py` file.
3. Run the main script `solution.py` and wait for its execution.
4. Change the log files names in the `__main__` function of the `plot.py` file and run it to generate figures.

4 Outcomes

At the end of the execution, several log files are created. The execution log includes details such as the generation number, the evaluated chromosome, and both the mean and standard deviation of the average storage and retrieval flow times. The generated files are:

- `log_all.csv`: All evaluated chromosomes.
- `log_generations.csv`: The survived population after every generation.
- `log_pareto_generations.csv`: The Pareto front of every generation.
- `log_pareto.csv`: The final Pareto front.

5 Files

5.1 `config.py`

This module defines the system’s operational boundaries, physical constants, and algorithmic constraints:

- **Kinematic variables:** Defines the operational constants for the handling equipment. All values are of the float type. The *Vehicles* are bounded by load time lv , column width dv , maximum velocity $vmaxv$, and acceleration av . The *Lifts* are bounded by load time ll , tier height dl , maximum velocity $vmaxl$, and acceleration al . The in/outflow *Conveyors* operates with a load time lc , aisle width spacing dc , and linear speed vc .
- **System exploration space:** Sets the system boundaries for exploration. All values are of the int type. These include the number of *Aisles* $\in [Aisles_min, Aisles_max]$, number of *Levels* $\in [Levels_min, Levels_max]$, number of *Columns* $\in [Columns_min, Columns_max]$, buffer capacities $Buf_Cap \in [Buf_Cap_min, Buf_Cap_max]$, and the number of available control policies for: storage N_SP , retrieval N_RP , lift N_LP , and vehicle N_VP .
- **Stochastic demand profiles:** Establishes the mean values for tote/order generation, using exponential distribution for inter-arrival times with mean iat_store and $iat_retrieve$. All values are of the float type.
- **Topological constraints:** Enforces strict storage capacity specification. A valid configuration must fall within the range $Slots \in [Slots_min, Slots_max]$. All values are of the int type.
- **Optimization and simulation parameters:** Defines the following parameters: population size POP_SIZE ; number of generations to be executed $NUM_GENERATIONS$; number of mutations and recombinations on every new generation $NUM_MUTATIONS$ and NUM_RECOMB ; number of independent runs for each candidate $REPETITIONS$; confidence level $confidence_level$; simulation stop criterion threshold $std_threshold$ and sliding window size $min_samples$. All values are of the int type, with exception of $confidence_level$ and $std_threshold$, which are of the float type.

5.2 `solution.py`

This script manages the high-level execution loop of the design space exploration. It handles file initialization, tracks solution metrics across generations, and coordinates the evolutionary pipeline. First, the initial generation is created. Next, the main loop executes the evaluation, mutation, recombination, and selection steps for a total of $NUM_GENERATIONS$. Each evaluation is repeated $REPETITIONS$ times, and their average and standard deviation are calculated. After each evaluation and generation, the resulting data is stored in the log files.

5.3 `genetic_algorithm.py`

This module implements the functions and operators of the genetic algorithm.

5.3.1 Initial_Pop()

Repeatedly samples values for each gene within its boundaries until it gathers *POP_SIZE* unique, feasible configurations or until it has reached *max_attempts*.

5.3.2 feasibility_check(c, min_s, max_s)

Checks if the candidate's topological dimensions meet the specification criterion of the minimum and maximum number of available storage slots. It returns a boolean value.

$$\min_s \leq \text{aisles} \times \text{levels} \times (\text{columns} - \text{buf_cap}) \times 2 \leq \max_s$$

Individual gene values are already restricted to their minimum and maximum limits during mutation and recombination, so they do not need to be checked here.

5.3.3 mutation(population, tested_chrom)

Starting from the existing population, this function repeatedly selects a random parent, mutates it into a new candidate, and retains the candidate if it is both novel and feasible. This process continues until it finds *NUM_MUTATIONS* valid offspring or until it has reached *max_attempts*. The following mutation operators are used:

- **Creep_Mutation_Dim(v, v_min, v_max, min_percent)**

Used for the physical dimensions *Aisles*, *Levels*, and *Columns*. This operator applies a variable modification that dynamically adjusts based on the current gene value relative to its boundaries (v_{min} and v_{max}) and the minimum percentage value allowed (*min_percent*). The core principle is that smaller values require a higher percentage step to effectively modify the parent's value (v_p).

$$\delta = 1 - \frac{v_p - v_{min}}{v_{max} - v_{min}} \cdot (1 - \text{min_percent})$$
$$v_c = \text{round}(v_p \cdot \text{uniform}(1 - \delta, 1 + \delta))$$

- **Creep_Mutation_Buf(v, v_min, v_max)**

Used for *Buf_Cap*. It adds an increment to the parent's value using a discrete uniform distribution over the set $\{-1, 0, 1\}$.

- **Random_Resetting(v_min, v_max)**

Used for the control policies. It generates a random integer within the index range of the available control policies.

5.3.4 recombination(population, tested_chrom)

Starting from the existing population, this function repeatedly selects two random parents, recombines them into a new candidate, and retains the candidate if it is both novel and feasible. This process continues until it finds *NUM_RECOMB* valid offspring or until it has reached *max_attempts*. The following recombination operators are used:

- **Arithmetic_Crossover(v1, v2)**

Used for the physical dimensions *Aisles*, *Levels*, *Columns*, and *Buf_Cap*. It calculates the average of the parent values.

- **Uniform_Crossover(v1, v2)**

Used for the control policies. It swaps control policies between parents with equal probability.

5.3.5 fast_non_dominated_sort_stochastic(fitness, std)

Ranks a population into Pareto fronts while considering the confidence interval of each candidate. The algorithm counts how many times each solution is dominated and lists which solutions it dominates. Solutions that are not dominated by any other solution form the first front. This front is then virtually removed, and the solutions that are now non-dominated form the next front. This process is repeated until all solutions are ranked. The next generation's offspring is determined by selecting individuals from the first fronts up to the population size limit. If the size of a single front exceeds the remaining available slots, the crowding distance metric is used to determine which candidates are added. It utilizes the following functions:

- **dominates_stochastic(avg1, avg2, std1, std2)**

Performs dominance testing between *avg1* and *avg2* considering their confidence intervals. These intervals are calculated using Student's t-distribution critical values (evaluated at a 95% confidence level over *REPETITIONS* – 1 degrees of freedom):

$$CI = \left[\mu - t_{\alpha/2, n-1} \frac{\sigma}{\sqrt{n}}, \mu + t_{\alpha/2, n-1} \frac{\sigma}{\sqrt{n}} \right]$$

where $n = \text{REPETITIONS}$, μ is the average, σ is the standard deviation, and α is the significance level. Candidate 1 dominates candidate 2 if and only if its upper bound strictly outperforms candidate 2's lower bound across all target objectives:

$$upper1 = avg1 + t_{\alpha/2, n-1} \frac{std1}{\sqrt{n}}$$

$$lower2 = avg2 - t_{\alpha/2, n-1} \frac{std2}{\sqrt{n}}$$

$$dominates \iff (\forall o \in \text{Obj}, upper1_o \leq lower2_o) \wedge (\exists o \in \text{Obj}, upper1_o < lower2_o)$$

- **crowding_distance(front, fitness)**

Measures solution density within the Pareto front to preserve diversification, favoring solutions in less dense regions to ensure a well-distributed spread of results.

5.3.6 evaluation(chrom)

Unpacks the chromosome structure, runs the simulation, and returns its performance metrics.

5.4 AVSRS.py

This module acts as the core simulation model. It maps physical assets, track geometries, and component logic using the PyCh process-oriented discrete-event modeling library.

5.4.1 Auxiliary functions

- **Experiment(...)**

Wraps the execution of individual model runs, capturing system logs and returning performance metrics to the genetic algorithm.

- **Model(...)**

Sets up the simulation run. It initializes the warehouse layout with a 50% initial item occupancy by pre-loading odd columns into *occupied_list* and leaving even columns in *empty_list*. It also instantiates the environment and the communication channels, and binds the processes. Finally, it triggers the simulation and return the performance metrics.

- `get_movement_time(distance, vmax, acc, load_time)`

Computes travel times for lifts and vehicles using physical kinematic models. It checks if the equipment can reach its maximum speed within the given travel distance before decelerating. The movement time is calculated as follows:

$$t = \begin{cases} 2 \cdot \sqrt{\frac{distance}{acc}} + load_time & \text{if } distance \leq \frac{v_{max}^2}{acc} \\ \frac{distance}{v_{max}} + \frac{v_{max}}{acc} + load_time & \text{if } distance > \frac{v_{max}^2}{acc} \end{cases}$$

- `get_conveyor_time(aisle)`

Calculates horizontal transport times for the conveyors using a fixed-velocity model:

$$t = \frac{(aisle + 1) \cdot dc}{vc} + 2 \cdot lc$$

5.4.2 class Tote

This class contains the information regarding the order. Its attributes are:

- `entrytime`: creation time of new orders using `env.now`.
- `id`: unique ID for each new order. Storage orders receive odd IDs, while retrieval orders receive even IDs.
- `aisle`: aisle coordinate.
- `tier`: tier coordinate.
- `column`: column coordinate.
- `side`: side coordinate, where Left side $\rightarrow 0$ and Right side $\rightarrow 1$.
- `is_storage`: boolean identifying whether the order is for storage or retrieval order. Retrieval $\rightarrow$ False, Storage $\rightarrow$ True.

5.4.3 Processes

5.4.3.1 Storage_Generator

Models the generation of storage totes that enter the system. Entities are created at random intervals, following an exponential distribution with mean `iat_store`, and held in an internal FIFO queue until they can be released downstream. To prevent the storage subsystem from overfilling, a new tote/order is only created if enough empty storage locations are currently available; otherwise, the process waits for the next generation attempt. Concurrently, whenever the queue is non-empty, the process offers to send the oldest queued tote through its output channel `c_out` to the *Storage_Controller*.

5.4.3.2 Retrieval_Generator

Models the generation of retrieval orders entering the system. Entities are created at random intervals, following an exponential distribution with mean `iat_retrieve`, and are held in an internal FIFO queue until they can be released downstream. A new retrieval order is only created if enough totes are currently in system; otherwise, the process waits for the next generation attempt. Concurrently, whenever the queue is non-empty, the process offers to send the oldest queued order through its output channel `c_out` to the *Retrieval_Controller*.

5.4.3.3 Storage_Controller

Coordinates the assignment and dispatch of incoming totes to their storage destinations within the racking system. When a new tote arrives through *c_store_order* from the *Storage_Generator*, the controller selects an available storage location using the configured assignment policy (*store_policy*), reserves it by removing the corresponding coordinates from *empty_list*, assigns them to the tote, and places it in a level-specific waitlist until space opens up on its destination aisle.

The buffer capacity at each aisle-level combination is tracked explicitly and capped at *buf_cap* slots; it is updated whenever a downstream process reports, through *c_inbound_changed*, that a tote has left the inbound buffer at that level. After every event, the controller checks each aisle for a free outgoing slot and, if one is available, pulls the next waiting tote from the lowest level that has both buffer capacity and a non-empty waitlist, offering it for dispatch on *c_store_to_conv* to the aisle *Conveyor* where the tote is to be stored.

5.4.3.4 Retrieval_Controller

Mirrors the *Storage_Controller*, but coordinates retrieval orders rather than incoming totes, operating at an (aisle, level) granularity rather than per-aisle. When a new order arrives through the *c_retrieve_order* from *Retrieval_Generator*, the controller selects an occupied storage location using the configured retrieval policy (*retrieve_policy*), reserves it by removing the corresponding coordinates from *occupied_list*, assigns them to the order, and places it in a level-specific waitlist until outbound buffer space becomes available.

The outbound buffer capacity is tracked per (aisle, level) combination, capped at *buf_cap* slots, and updated whenever a downstream process reports, through *c_outbound_changed*, that a tote has left the outbound buffer at that level. Unlike the *Storage_Controller*, each (aisle, level) pair here has its own independent dispatch slot, allowing multiple retrievals to be in transit simultaneously within the same aisle, with each being offered for dispatch on its corresponding channel in *c_retrieve_to_dem* for the specific *Demand_Buffer*.

5.4.3.5 Conveyor

Models a single-capacity conveyor segment that transports one tote at a time between *Controllers* and *Lifts*. The process repeatedly receives a tote through *c_in*, holds it for a travel time determined by *get_conveyor_time(tote.aisle)*, and then forwards it downstream through *c_out*. The communication channels used depend on the flow direction and the specific aisle involved.

5.4.3.6 Demand_Buffer

Coordinates the assignment and dispatch of the *Vehicle* based on storage and retrieval orders. Orders arriving from either the *buffer* via *c_buf_to_dem* or from the *Retrieval_Controller* via *c_retrieve_to_dem* are appended to the waiting list as they arrive. Whenever that list is non-empty, the process offers to send a chosen tote, selected by the configured vehicle dispatch policy (*vehicle_policy*), through *c_dem_to_veh*. Once this transmission succeeds, the dispatched tote is removed from the waiting list, and the tracked column is updated to match it, ensuring the next dispatch decision accounts for the vehicle's new position.

5.4.3.7 Vehicle

Models the physical movement of a tier-captive vehicle that executes both storage and retrieval moves within an aisle after receiving its next assignment from the *Demand_Buffer* through *c_dem_to_veh*.

For a storage move, the vehicle first travels to the buffer (modeled as *buf_cap* columns beyond column 0), requests the specific tote from the *Buffer* via *c_selected_tote*, receives it through

c_buf_to_veh, and then travels from the buffer to the tote's assigned column to place it on the rack. Finally, the slot's coordinates are appended to *occupied_list* and the storage flow time is reported to the *Storage_Monitor* through *c_store_done*.

For a retrieval move, the vehicle travels directly to the tote's designated column, picks it from the slot, and travels onward to the buffer to hand it off to the *Buffer* through *c_veh_to_buf*. Once complete, the slot's coordinates are appended to *empty_list*.

5.4.3.8 Lift

Models the physical movement of a lift that connects the aisle's conveyor to the tiers. In every iteration, the lift simultaneously offers to receive a tote that needs storing (*c_conv_to_lift*) and advertises its own availability for a retrieval (*c_lift_ready*).

If a storing tote arrives, the lift travels down to the ground level to collect it, moves up to its destination tier, delivers it to that tier's inbound buffer via *c_lift_to_buf*, and updates its current position accordingly.

If its readiness signal is accepted instead, it waits to be informed by the *Buffer* (which coordinates the lift policy) which tier to service via *c_next_tier*. It then travels there, takes the tote from the outbound buffer through *c_buf_to_lift*, descends to the ground level, and hands the tote off to the conveyor through *c_lift_to_conv*. Finally, the lift reports its current tier through *c_lift_position*.

5.4.3.9 Buffer

Encompasses all inbound and outbound buffers of a single aisle. On the inbound side, totes arriving from the *Lift* via *c_lift_to_buf* are queued per level, capped at *buf_cap*, and a separate *to_notify* queue tracks which of them still need to be announced to that level's *Demand_Buffer* through *c_buf_to_dem*. Once a *Vehicle* later requests a specific tote through *c_selected_tote*, it is removed from *inbound* and handed over via *c_buf_to_veh*, and *c_inbound_changed* reports the freed slot back to the *Storage_Controller*.

On the outbound side, totes dropped off by a *Vehicle* through *c_veh_to_buf* are similarly queued per level, also capped at *buf_cap*. Whenever any level has waiting totes, the process accepts the Lift's readiness signal on *c_lift_ready*, selects which level to service next via the configured *lift_policy*, and hands off both the destination tier (*c_next_tier*) and the tote itself (*c_buf_to_lift*) before reporting the freed outbound slot to the *Retrieval_Controller* via *c_outbound_changed*.

5.4.3.10 Storage_Monitor

Continuously collects completed storage flow time samples reported by *Vehicle* instances through *c_store_done*, using them to update the mean and the standard deviation (*stats* dictionary). It also tracks a running average throughput and implements a steady-state check with a sliding window of *min_samples* observations. It triggers the *stable_done* event when the standard deviation drops below a *std_threshold*.

5.4.3.11 Retrieve_Monitor

Continuously collects completed retrieval flow time samples whenever a tote is received from any *Conveyor* via *c_retrieve_done*, using them to update the mean and the standard deviation (*stats* dictionary). It also tracks a running average throughput and implements a steady-state check with a sliding window of *min_samples* observations. It triggers the *stable_done* event when the standard deviation drops below a *std_threshold*.

5.5 control_policies.py

Collection of control policies for storage, retrieval, lift, and vehicle. The policies, with their IDs are:

- **store_policies and retrieval_policies:**
 0. `get_random_idx`: Selects a location coordinate at random.
 1. `get_FCFS_idx`: Follows a first-come, first-served sequence.
 2. `get_closest_idx`: Prioritizes the nearest aisle, breaking ties by lowest tier and closest column.
 3. `get_emptiest_fullest_tier_idx`: Target tiers with extreme load profiles (emptiest for storage, fullest for retrieval), breaking ties at random.
 4. `get_lowest_tier_idx`: Prioritizes lower vertical positions, breaking ties by FCFS.
 5. `get_closest_column_idx`: Prioritizes closer horizontal positions, breaking ties by FCFS.
- **lift_policies:**
 0. `get_tier_fcfs`: Follows a first-come, first-served sequence.
 1. `get_tier_shortest`: Targets the nearest non-empty tier, breaking ties by lowest tier.
 2. `get_tier_farthest`: Targets the farthest non-empty tier, breaking ties by highest tier.
 3. `get_tier_most_loaded`: Targets tiers with the most totes waiting, breaking ties by lowest tier.
 4. `get_tier_least_loaded`: Targets tiers with the fewest totes waiting, breaking ties by lowest tier.
 5. `get_tier_lowest`: Targets the lowest non-empty tier.
 6. `get_tier_highest`: Targets the highest non-empty tier.
 7. `get_tier_weighted`: Targets the non-empty tier with the highest score calculated as the ratio of buffer occupancy to physical travel distance $\frac{quantity}{distance+1}$, breaking ties by lowest tier.
- **vehicle_policies:**
 0. `get_column_fcfs`: Follows a first-come, first-served sequence.
 1. `get_column_lcls`: Follows a last-come, first-served sequence.
 2. `get_column_shortest`: Targets the column closest to the vehicle's current position, breaking ties by FCFS.
 3. `get_column_farthest`: Targets the column farthest from the vehicle's current position, breaking ties by LCFS.
 4. `get_column_retrieval_first`: Targets the column closest to the vehicle's current position while prioritizing retrieval orders, breaking ties by FCFS.
 5. `get_column_storage_first`: Targets the column closest to the vehicle's current position while prioritizing storage orders, breaking ties by FCFS.

5.6 PyCh_architecture_diagram.pdf

The architecture is organized hierarchically into three main operational layers: the Global System Layer, the Aisle Layer, and the Tier Layer. Communication across these layers relies on dedicated physical and logical channels as described previously.

5.7 plot.py

Provides three plotting functions for visualizing and comparing simulation outputs, all operating on `.csv` log files. All three functions render a scatter plot of the average storage flow time against the average retrieval flow time. The available functions are:

- `plot_single_file`
Plots a single dataset. This is useful for inspecting the Pareto front produced by a specific experimental setup.
- `plot_two_files`
Overlays two sets of results using distinct markers and colors, allowing for a direct visual comparison between two experimental setups.
- `plot_comparison`
Overlays two sets of results, explicitly indicating the stochastic dominance relation between them. Each sample in each solution is tested against every sample in the other, and the four resulting subsets are rendered with distinct colors and markers, showing each solution's dominated and non-dominated points.

These functions accept the following parameters:

- `file names`: path to the log files.
- `subtitle`: string appended to the fixed title prefix "Average storage flow time x Average retrieval flow time —", allowing each plot to be identified by its experimental setup.
- `xlim, ylim`: optional tuples (*min*, *max*) that fix the axis ranges. When set to `None`, `matplotlib` handles the scaling automatically.
- `labels`: strings used in the legend of the scatter plots.